APGR1+3 2026: Evaluation Table for Task 1 (Required Submission) **Yaroslav Usatenko**

		Fulfilled (0 % - 100 %)	Control Solution (Key or GUI)	Any comments on the request
Line segment	Correct rasterization in all quadrants	100 %	L / GUI	Bresenham; all octants and reversed endpoints.
	Mark the line rasterization algorithm in a code comment	100 %	Code	Algorithm is named in BresenhamLiner comment.
	Interactive endpoint input	100 %	Mouse	Arbitrary endpoints by press-drag-release.
	Elastic rendering (redrawing when dragging the mouse)	100 %	Mouse drag	Live temporary line is redrawn while dragging.
	Custom Class Implementation and Inheritance	100 %	Code	Abstract Liner with BresenhamLiner implementation.
Polygon	Representation and Encapsulation in a Class	100 %	P / GUI	Polygon class encapsulates a vertex list.
	Input and flexible boundary rendering	100 %	Mouse drag	Interactive vertices with live flexible boundary preview.
	Closing a polygon	100 %	Automatic	Last vertex is connected back to the first.
Aligned line (Shift key)	Horizontal	100 %	Shift + drag	Nearest horizontal direction.
	Vertical	100 %	Shift + drag	Nearest vertical direction.
	Diagonal	100 %	Shift + drag	Nearest +45 / -45 degree diagonal.
	Interactive mode input and switching when dragging the mouse	100 %	Shift + drag	Constraint changes live from current mouse position.
Seed Fill – raster defined boundary	Specifying the origin	100 %	F / B + click	Seed origin is selected directly on the canvas.
	Encapsulation	100 %	Code	SeedFill class encapsulates both seed rules.
	Border color constraints	100 %	B / GUI	Boundary-restricted fill stops at boundary color.
	Background color restrictions	100 %	F / GUI	Background-restricted fill follows background color only.
Scan Line – vector-defined boundary	Correctness of filling	100 %	S / GUI	Scan-line fill supports convex and concave polygons.
	Encapsulation	100 %	Code	ScanLineFill class encapsulates the algorithm.
	Outline with border color	100 %	S / GUI	Polygon outline is redrawn with boundary color.
	Implementation of the intersection sorting algorithm	100 %	Code	Intersections are ordered by explicit insertion sort.
Clipping a Polygon	Correct convex polygon clipping even for non-convex and self-intersecting polygons	100 %	X / GUI	Sutherland-Hodgman; convex clip, concave/self-intersecting subject supported.
	Editing a Clipping Polygon	100 %	V / Edit Points	Drag clipping vertices; right-click a vertex to delete it.
	Polygon orientation selection	100 %	O / Reverse Clip	Switch CW/CCW orientation; clipping also detects either orientation.
Delete structures and canvas with the C key		100 %	C / Clear	Clears raster, lines, polygons, fills and clipping data.

APGR1+3 2026: Evaluation Table for Task 1 (Required Submission) **Yaroslav Usatenko**

GIT repository with the resulting code Please provide **a link** to the repository		100 %	GitHub	https://github.com/wharyladice/APGR_Task1
Bonus	Scan Line Pattern Fill	100 %	T + S / GUI	PatternFill interface with repeating checker pattern.
	Seed Fill Pattern Fill	100 %	T + F/B / GUI	Same PatternFill interface is used by Seed Fill.
	Editing the Position of line endpoints	100 %	E / Edit Lines	Drag either endpoint of an existing line.
	Editing and deleting polygons its points	100 %	V / Edit Points	Drag vertices; right-click deletes subject or clipping vertices.
	Implement the Seed Fill algorithm using a queue or stack	100 %	Code	Seed Fill uses ArrayDeque as an explicit stack.
	Regular GIT commits	0 %	Git	Not claimed; repository currently contains one commit.
Custom extensions	Toolbar and color selection	100 %	GUI	Swing toolbar and standard JColorChooser.
	Safe closed-region Seed Fill	100 %	F / B	Open regions reaching the canvas edge are rejected.
	Automatic one-shot Seed mode	100 %	F / B	After fill/cancel the previous drawing mode is restored.
	Editing handles and status overlay	100 %	E / V	Visible edit handles plus current mode/status instructions.
Use of AI	Indicate which, where, and how AI tools were used in the development of the project	100 %	AI was used only for minor assistance with debugging, mainly to identify syntax errors, as well as for code review and reviewing the project documentation.	